

workflows. The Framework team functions with autonomy to drive good quality within our Reference Architectures and GET above all else.

Some optimal success criteria include

1. A decrease in the cost of support per customer
1. Lower time to value for new installations
1. More instances on the recent version of GitLab
1. Fewer upgrade-related incidents
1. Instances on supported architecture

Seeing that many of these metrics are owned by multiple teams, the Framework Team will contribute here but does not have full agency to drive.

What does it mean to Operate GitLab?

GitLab is a complex application. Organizations typically have a dedicated team to manage GitLab internally. This operation includes day-to-day maintenance, upgrades, backup and restore, and downtime. This team's goal is to make this operation easy for all organizations, including our own deployments.

It is not solely the responsibility of the Framework team to make GitLab easy to operate, but to provide guidance to the wider GitLab organization to develop in a way that is conducive to a good user experience for admins. Managing GitLab components in a tested and supported architecture significantly lowers the operational aptitude required.

Delivery of Mission

1. Easy enablement materials to consume reference architectures for account teams and operators
2. Collaborate on structured guidance for operational excellence in feature development
3. Verify upgrade paths
4. Stewards of a good experience for managing GitLab with GET and the Reference Architectures

Tools and Responsibilities

The Framework team owns and leads our GitLab Environment Toolkit and Reference Architectures initiatives, which are key programs with the aim assist and enable customers to deploy and maintain their GitLab environments. This includes external users as well as easing the operational requirements for our own SaaS offerings.

GitLab Environment Toolkit (GET)

GET is a set of opinionated Terraform and Ansible scripts to assist with deploying scaled self-managed GitLab environments following the Reference Architectures. A good experience with GET means users will more quickly have an environment (lower TTV) and long-term success (decreased need for support).

Reference Architectures

The reference architectures will provide the best user experience with the intent to decrease

the amount of required support from GitLab. The Framework team will continue to ensure the RAs meet the needs of customers by improving architecture designs over time. The Framework team will also provide guidance for internal teams to review and add components as needed.

Upgrade Path Validation

Upgrading GitLab is one of the most challenging aspects of its operation and often requires dedicated attention to effectively plan the upgrade process. The Framework team continuously validates the upgrade paths between releases to ensure that only necessary, planned stops are introduced, thereby instilling confidence in the upgrade process for our Self-Managed and Dedicated customers.

Other Responsibilities

- 1. Architecture performance and non-functional testing
- 1. Enablement of GitLab account teams to provide appropriate recommendations
- 1. Self-serve experience for scaling GitLab

Team Members

{{< team-by-manager-slug "Isogunle" >}}

Product Manager: Martin Brümmer

Roadmap

The team's primary roadmap is tracked within the Roadmap epic. Our roadmap is aligned with GitLab Delivery's overall strategic objectives and is reviewed and updated as needed.

Working with Us

To engage with the Framework team, please refer to the following.

Please kindly note that as an engineering team our resources are finite. Direct asks to join customer calls or projects are not within the remit of this team and should instead be directed accordingly to Product Management, Support or Professional Services accordingly.

Slack Channels

The following list includes Slack channels relevant to projects within the team's scope:

gsoftwaredelivery-framework - This is the primary Slack channel for discussions related to the GitLab Delivery Framework team.

gitlabenvironmenttoolkit - For discussions, questions, and requests specific to the GitLab Environment Toolkit (GET).

reference-architectures - For discussions, questions, and requests related to Reference Architectures.

Our slack group handle is @software-delivery-framework-team.

Reference Architecture Review requests

For any requests relating to customer environments, either proposed or existing, they must be raised in the Reference Architectures project in the following circumstances only with the given templates:

environment-review-request - Sanity checks of new environment designs, not already covered in the Reference Architecture docs.

request-for-help - Help requests from the Support Team for assistance when an issue is suspected to be due to the environmental design.

Requests should be opened two or more business days before action is needed to ensure the team has time to prepare and we kindly ask for this process to be followed for tracking and capacity reasons.

It is important to note that only requests following the outlined process will be addressed. To ensure effectiveness and minimize disruption, we strongly discourage directly tagging team members in real-time on issues or customer requests.

Reference Architecture consultations will not include the following:

1. No involvement in direct customer calls or communications.
1. All requested data stated in the template must be proactively supplied.

Environment Build Requests

The Framework team will not fulfill bespoke environment build requests due to capacity constraints. Also, building large sandbox environments with GET should be generally discouraged due to cost implications unless strictly required. However, here are some options to self-serve in building these types of environments:

Self-service using GET documentation

Teams can follow the comprehensive GET documentation to build their own environments. This is the primary recommended approach for teams that need custom environments.

Using the automated Sandbox Cloud offering is recommended for setting up a cloud account for testing and development purposes.

Run GitLab locally: For simpler needs, running GDK or Docker locally is recommended instead of building large environments to reduce costs and complexity.

Use shared environments from Infra: Utilizing existing shared environments provided by the Infrastructure team.

Triage DRI

The Triage DRI is a team role that acts as an initial point of contact for ad hoc team requests, triaging pipelines and redirecting inquiries to appropriate team members. The role enables timely responses while spreading knowledge across the team.

This is a limited role targeting initial triage. It is not a replacement for any initiative maintainers or DRIs as well as not a replacement for Support or other times.

Responsibilities

1. Monitoring and triaging the Reference Architecture performance pipelines
1. Monitoring and responding to requests in GET/RA Slack channels.

Responses are aimed to be timely, with a general target of a few days at most. This is not a strict target and can vary depending on team availability.

Process Guidelines

DRI maintains ownership of cases they begin during their rotation, even after the rotation ends

DRI focuses solely on new incoming cases

Questions requiring specific expertise will be redirected to appropriate team members - when in doubt, seek assistance from the initiative's maintainers.

Shadows

The Triage DRI may have a shadow assigned for learning purposes and onboarding to become a DRI themselves.

The shadowing process works as follows:

Shadowing is primarily asynchronous

Progress is tracked through dedicated shadowing issues, where debriefs and questions about specific instances should be raised

Minimum goal of four rotations but this can continue until such a time when the engineer feels comfortable taking on the DRI role

Cadence

The Triage DRI role will be on a weekly rotation cadence.

Please see the schedule [here](#).

Project Management

<!-- List of Trackers and scope -->

Framework Issue Tracker.

GitLab Environment Toolkit (GET) issues - Primary tracking system for the GitLab Environment Toolkit. This board encompasses requests, enhancements, technical debts items and customer reported issues.

GitLab Upgrades Test Coverage - This epic encompasses issues related to testing upgrade paths.

Reference Architecture issues - This is where we track work related to Reference Architectures and conduct reviews for customer environments upon request.

Epics

<!-- Top level Epic(s) -->

This top-level epic monitors the different initiatives and sub-epics within the team and their respective statuses. It is populated automatically through epic automation.
Every sub-epic must

1. Epics must be created with the epic template linked below.
1. Have the top-level epic as the parent epic.
1. Have a completion date and a DRI for providing a weekly status update.
1. Have a relevant workflow:: or workflow-infra:: label.
1. Epic DRIs will be pinged on Tuesdays to leave a status update for the week in the epic.
1. These updates are collected into the team's top-level epic which feeds into the Grand Review.

We follow Platforms Project Management practices as outlined.

Labels

<!-- Labels explanations -->

The primary team label is group::framework. This label should be applied to issues and epics throughout the organization that require the team's attention.

The team tracks work using the workflow and workflow-infra labels across issues, epics, and MRs. Team members will ensure that their work items are updated with the correct workflow labels.

```
| gitlab-org issues | gitlab-com/gi-infra issues |  
| ----- | ----- |  
| ~"workflow::refinement" | ~"workflow-infra::Triage" |  
| ~"workflow::ready for development" | ~"workflow-infra::Ready" |  
| ~"workflow::in dev" | ~"workflow-infra::In Progress" |  
| ~"workflow::in review" | ~"workflow-infra::Under Review" |  
| ~"workflow::blocked" | ~"workflow-infra::Blocked" |  
| ~"workflow::verification" | ~"workflow-infra::Verify" |  
| ~"workflow::complete" | ~"workflow-infra::Done" |
```

Epic Template

<details><summary>Epic Template</summary>

markdown

DRI :levitate:

- TBC

Participants :bustsinsilhouette:

Problem to solve :thinking:

Why :results-for-customers:

<!-- Explain the importance of this epic and its contributions to delivering results for customer. -->

Future Maintenance :construction:

<!-- If this involves building a new piece of tooling, pipelines, or artifacts, clearly identify and align on future maintenance responsibilities. -->

Documentation :book:

Publicly Accessible Documentation:

Links / references :books:

<!-- STATUS NOTE START -->

<!-- STATUS NOTE END -->

/label ~"group::framework" ~"workflow-infra::Triage"

</details>

Team Impact

The following are records of the annual achievements of the Framework team.

1. Team Impact Overview for 2024

SECTION: engineering/infrastructure-platforms/gitlab-delivery/framework/reference-architecture-validation-testing.md

The GitLab Delivery: Framework team maintains and validates the GitLab Reference Architectures - production-ready environment designs for deploying GitLab at scale. This documentation details the testing process that ensures these architectures meet performance standards.

Overview

The GitLab Reference Architectures undergo comprehensive testing to validate their performance, reliability, and scalability. The validation process involves multiple tools and automated pipelines that continuously test these architectures against the latest GitLab releases in efficient fashion.

Testing Tools and Infrastructure

GitLab Environment Toolkit

The GitLab Environment Toolkit is a set of opinionated Terraform and Ansible scripts for deploying GitLab at scale. For our testing process, GET provides:

- Automated deployment of all Reference Architecture sizes (1k to 50k)
- Support for both Linux package and Cloud Native Hybrid architectures
- Consistent environment provisioning across GCP and AWS
- Built-in monitoring setup for performance validation
- Support for nightly builds testing
- Standardized configuration management aligned with Reference Architectures

GitLab Performance Tool

The GitLab Performance Tool is our primary testing framework that measures server performance under load. Key capabilities include:

- Automated performance testing of critical GitLab endpoints
- Customizable load scenarios based on real-world usage patterns
- Detailed performance metrics and reporting
- Integration with CI/CD pipelines

For detailed information about current test scenarios, see the [Test Details](#) wiki page.

Test Process

Our testing process runs continuously against various Reference Architectures sizes across the main Cloud Providers using GitLab's latest or nightly releases, enabling early detection and remediation of performance issues.

1. Environment Preparation

- Deploys or updates target environment using test environment config
- Installs latest release package depending on environment type
- Ensures consistent test conditions
- Validates environment setup and health

2. Performance Testing

- Executes standardized test suite covering:
 - API endpoints
 - Web interfaces
 - Git operations (pull/push)
- Measures key metrics:
 - Response times
 - Throughput
 - Resource utilization
 - Error rates

3. Results Processing

- Publishes detailed results to Reference Architectures Wiki
- Posts summaries to ref-arch-performance-tests Slack channel
- Flags performance regressions for review

4. Environment Management

- Stops test environment instances
- Optimizes resource usage and costs
- Maintains environment state for next test cycle

Testing Methodology

Environment Isolation

We maintain dedicated test environments to ensure reliable and consistent results:

1. Control over Test Conditions

- Eliminates interference from shared workloads
- Ensures consistent test data and configuration
- Enables accurate performance measurement

2. Resource Management

- Prevents conflicts with other testing pipelines
- Allows full environment access for troubleshooting
- Maintains consistent resource availability

Test Coverage

Our testing program includes:

- Daily and weekly testing of Linux package environments (all sizes) across GCP and AWS
- Weekly testing of Cloud Native Hybrid configurations on GCP and AWS
- Ad hoc validation of new architecture variations being explored
- Testing of cloud provider service integrations

SECTION: engineering/infrastructure-platforms/gitlab-delivery/self-managed/_index.md

Overview

Self Managed team is focused on installation and upgrade mechanisms to ensure smooth deployments. This includes system integration, scripting, templating, and related configuration management tooling.

Team members

The following people are members of the team:

{{< team-by-manager-slug "plu8" >}}

Working with the community

The installation and upgrade process is the first feature that all system administrators experience when working with GitLab.

As a result, the projects managed by the Self Managed team have a high level of engagement by the user-base. The GitLab community is made up of more than just code contributors; users logging issues and feature requests are constantly pushing us forward and helping create a better experience.

We strive for the following in our public projects:

1. Uphold our Community Code of Conduct.
1. Enable GitLab's mission that everyone can contribute..
1. Show our work in public.
1. Recognize and thank contributors for their work.
1. Respect contributors donated time by providing a timely review turnaround time.

Working with Open Source communities

The open core of GitLab is built on top of thousands of open source dependencies. These dependencies and their communities are important to the GitLab strategy, and working with these dependencies is an essential part of the projects the team maintains.

We strive to:

1. Consider the impact of our work on the open source communities that we benefit from.
2. Promote the importance of these open source communities within GitLab.
3. Raise issue with any decision that goes against our stewardship promises.
4. Find opportunities to contribute back the changes we make.

Public by default

All work carried out by the team is public. Some exceptions apply:

- Work has possible security implications - If during the course of work security concerns are no longer valid, it is expected for this work to become public.
- Work is done with a third party - Only when a third party requests that the work is not public.
- Work has financial implications - Unless financial details can be omitted from the work.
- Work has legal implications - Unless legal details can be omitted from the work.

Some of the team work is carried out on our development server at dev.gitlab.org. Infrastructure overview document lists the reasons.

Unless your work is related to the security, all other work is carried out in projects on GitLab.com.

If you need to submit a sensitive issue, please use confidential issues.

If you are unsure whether something needs to remain private, check with the team Engineering Manager.

Work/life harmony

Working all-remote and asynchronous first offer flexibility in how team members approach their workday. Team members must make choices on how best to balance work time with other areas of life.

For new team members, the following resources provide examples on how to focus their time:

- How team members approach their day
- Blog post: A day in life of a remote worker
- The option of a non-linear workday
- GitLab handbook: Work/life Harmony

The following GitLab Handbook areas are key in maintaining a healthy work/life balance.

- Family and Friends First, work second
- Combating burnout, isolation, and anxiety in the remote workplace
- Recognizing Burnout

SECTION: engineering/infrastructure-platforms/gitlab-delivery/self-managed/workflow.md

Overview

The Self Managed team is responsible for installation, upgrades, scaling, migration, and configuration across various deployment methods. Our work often involves:

1. Responding to both internal and external customer requests
2. Supporting multiple deployment methods and platforms simultaneously
3. Managing technically complex integration work with varied timelines
4. Balancing planned feature development with maintenance work

Following Kanban) approach provides these benefits:

- Improved flow efficiency - Work items progress continuously rather than being tied to milestone boundaries
- Greater adaptability - Easier to respond to urgent deployment issues or customer requests without disrupting planned work
- Better visualization of workload - Clearer view of in-progress work across different deployment platforms
- More predictable delivery - Work completion is based on team capacity rather than milestone deadlines

Kanban Implementation

We also use the GitLab product development flow and labels in principle. However, we usually skip below phases due to the nature of our work:

- Validation phase 3: Design
- Validation phase 4: Solution Validation

Workflow Diagram

mermaid

flowchart LR

classDef backlog fill:f9f9f9,stroke:999

classDef refinement fill:fce8d4,stroke:e67e22

classDef ready fill:d4f6c8,stroke:2ecc71

classDef development fill:d4e8f6,stroke:3498db

classDef readyreview fill:b3d1f0,stroke:2980b9

classDef review fill:e0d4f6,stroke:9b59b6

classDef done fill:e8e8e8,stroke:666

classDef blocked fill:ff9999,stroke:e74c3c,stroke-width:2px,stroke-dasharray: 5 5

Backlog([backlog]):::backlog --> Refinement([workflow::refinement]):::refinement

Refinement --> Ready([workflow::ready for development]):::ready

Ready --> Development([workflow::in dev]):::development

Development --> ReadyReview([workflow::ready for review]):::readyreview

ReadyReview --> Review([workflow::in review]):::review

Review --> Done([workflow::complete]):::done

Blocked([workflow::blocked]):::blocked

Refinement -. -> Blocked

Ready -. -> Blocked

Development -. -> Blocked

ReadyReview -. -> Blocked

Review -. -> Blocked

Blocked -. -> Refinement

Blocked -. -> Ready

Blocked -. -> Development

Blocked -. -> ReadyReview

Blocked -. -> Review

LookLeft["Look Left
(Daily)"] -. -> Refinement

LookRight["Look Right
(Daily)"] -. -> ReadyReview

LookRight -. -> Review

WeeklySync["Weekly Sync
(Weekly)"] -. -> Backlog

WeeklySync -. -> Ready

WeeklySync -. -> Development

BlockedReview["Blocked Issues Review
(Daily Priority)"] -. -> Blocked

subgraph WIP Limits

Refinement -. - RL["2 per engineer"]

Ready -. - RDL["1.5× team size"]

Development -. - DL["2 per engineer"]

ReadyReview -. - RRL["2× team size"]

Review -. - RVL["1.5× team size"]

end

subgraph Ceremonies

BlockedReview

```
LookLeft
LookRight
WeeklySync
end
```

Kanban Board Structure

The Self Managed Kanban board would use the following columns, aligned with GitLab's product development workflow:

Column	Description	WIP Limit
backlog	Prioritized but not ready for development	No limit
workflow::refinement	Issues being refined, broken down	2 per engineer
workflow::ready for development	Fully specified, ready to be worked on	1.5× team size
workflow::in dev	Currently being implemented	2 per engineer
workflow::ready for review	Implementation complete, waiting for reviewer to pick up	2× team size
workflow::in review	Under active code review	1.5× team size
workflow::complete	Completed work	No limit
workflow::blocked	Work that cannot proceed due to some reason	Tracked but no limit

Key Ceremonies

Daily Activities

- Look Left & Right Before Dev - Help move one item forward from refinement or review (ready for review/in review) before starting new development
- Async Stand-up - Brief asynchronous check-in focused on flow and impediments

Weekly Meetings

1. Team Sync (Weekly)

- Review Kanban board progress with EM and PM
- Discuss impediments and prioritization
- Ensure alignment with product goals

2. Team Demo (Weekly)

- Sharing with Build group
- Regular demonstrations of completed work
- Sharing knowledge about new features or improvements
- Gathering feedback from team members and stakeholders
- Increasing visibility of the team's accomplishments

3. Maintainer Discussion (Weekly)

- Sharing with Build group
- Regular review of merge requests and code quality
- Discussion of technical decisions and architecture
- Coordination between maintainers on critical components

- Ensuring consistency in code standards and practices

4. Engineers Huddle (On demand)

- Development-focused discussion
- Implementation strategies and technical decisions

Monthly Activities

On the last week's weekly sync meeting every month.

- Flow Retrospective - Review flow metrics and process improvements
- Release Planning - Align with next milestone planning using flow-based priorities

Quarterly Activities

- Roadmap planning - Participate in quarterly roadmap planning to align flow with strategic objectives
- Retrospective - Reflect on the past quarter's achievement, identify areas for enhancement, and create action plans for the future

Priority definition

Self Managed team uses infrastructure-wide priority labels to determine the position in the Kanban columns.

Priority	Label	Position in Kanban	Action Required
1	~priority::1	Top of active columns	Immediate attention required
2	~priority::2	High in active columns	Address after Priority 1 items
3	~priority::3	Middle of workflow	Address after higher priorities
4	~priority::4	Bottom of workflow	Address when resources available

- "Active columns" refer to the workflow stages where work is currently being processed. These include:

- Ready for Development: Issues prioritized and ready to be worked on
- In dev: Issues currently being worked on by team members
- Ready for review: Work completed but waiting to be reviewed
- In review: Work completed but going through review and approval

- "Middle/Bottom of workflow" refers to issues with ~priority::3 and ~priority::4 that are positioned in the central part of our board. These items:

- Have moderate urgency
- Are positioned below Priority 1 and 2 items in active columns
- Are typically scheduled after more urgent work
- May be in any workflow stage (refinement, development, review)
- Represent important work that should be completed, but can wait until higher priorities are addressed

- The position in Kanban board needs to be managed and maintained manually due to the issue boards' limitation.

Work Prioritization Guidance

1. Highest Priority First: Always address higher priority items before any lower priority ones
2. Milestone/Deadline-Driven Work: Issues with fixed deadlines or milestone take precedence over non-deadline work of the same priority
3. Dependency Resolution: Address blocking issues before dependent issues regardless of initial priority
4. WIP Limits: Maintain work-in-progress limits to ensure focus on completing high-priority items

Issue Weight Definition

Weight	Extra investigation	Surprises	Collaboration	Description	Examples
1: Trivial	not expected	not expected	not required	Cannot benefit from further breakdown	

- Create or update simple documentation
- Fix a missing encoding problem in secret management

2: Small	possible	possible	possible	Simple tasks with clear requirements	
----------	----------	----------	----------	--------------------------------------	--

- Update to a new version of key dependency
- Create or update complex documentation

3: Medium	likely				
-----------	--------	--	--	--	--

likely
likely
More complex tasks requiring coordination

- Add E2E test to a CI pipeline
- Addressing security vulnerabilities in dependencies with our customization

5: Large
highly likely
highly likely
highly likely
Consider breaking down if technically feasible

- Migrate secret generator module to the new framework
- Introduce a new application API and custom resource definition to Operator V2

8+
guaranteed
guaranteed
guaranteed
Too large - must be broken down into multiple issues and grouped into an Epic

- Address DockerHub pull limits across projects
- Self managed: Support Rollout of Container Registry for Self-Managed Instances

1. Notes on weights:

- Weights are contextual and can be affected by domain knowledge, experience levels, and time at GitLab
- Weights are not set in stone and can be adjusted by the author/assignee if an issue requires more effort than initially estimated
- For Weight 5 issues, team members are encouraged to discuss whether breaking it down would be beneficial
- Weight 8+ issues must be broken down and should not be marked as ready for development

2. WIP Limits Implementation

- Set column WIP limits based on item count initially, review once there is more data

Flow Metrics

Use these flow-based metrics instead of milestone-based completion metrics, we will establish the baseline metrics and aim for steady improvement by FY27Q1.

```
mermaid
graph LR
    classDef metric fill:d4e8f6,stroke:3498db

    subgraph Kanban Metrics
        CT[Cycle Time]:::metric
        LT[Lead Time]:::metric
        FE[Flow Efficiency]:::metric
        TP[Throughput]:::metric
        WIP[WIP Ratio]:::metric
        WIA[Work Item Age]:::metric
    end

    subgraph Timeline
        B[Backlog] --> R[Refinement] --> RD[Ready] --> D[Development] --> RV[Review] --> C[Complete]
    end

    CT -. -> D
    CT -. -> C
    LT -. -> B
    LT -. -> C
    FE -. -> Timeline
    TP -. -> C
    WIP -. -> D
    WIP -. -> RV
    WIA -. -> Timeline
```

1. Cycle Time

Definition: The total elapsed time from when work begins on an issue until it is delivered to production.

Components:

- Code Time: Time spent coding the solution
- Review Time: Time spent in code review (MR open · MR merged)

Target: Our target cycle time varies by work complexity (To be updated by FY27Q1):

- Trivial changes (Weight 1): $\leq x$
- Small changes (Weight 2): $\leq x$
- Medium changes (Weight 3): $\leq x$
- Large changes (Weight 5): $\leq x$

Measurement: (To be updated by FY27Q1)

2. Lead Time

Definition: The total elapsed time from when an issue is created until it is delivered to production.

Components:

- Planning Time: Time from issue creation to development start
- Cycle Time: (as defined above)

Target: Our target lead times are (To be updated by FY27Q1):

- Priority 1 issues: $\leq x$
- Priority 2 issues: $\leq x$
- Priority 3 issues: $\leq x$
- Priority 4 issues: $\leq x$

Measurement: (To be updated by FY27Q1)

3. WIP Ratio

Definition: The ratio of work items in progress compared to team capacity. Calculated as the number of active work items divided by the number of team members.

Target :

- Optimal WIP ratio: 1-3
- Warning threshold: ≤ 3.0
- Critical threshold: ≤ 4.0

Measurement: Count of issues in the ~"workflow::in dev", and ~"workflow::in review" stages divided by the number of active team members.

4. Throughput

Definition: The number of work items (issues) completed per time period.

Target: Teams should establish their baseline throughput and aim for steady improvement.

Measurement: Count of issues moved to the "closed" state per week.

5. Work Item Age

Definition: The age of currently open work items.

Target:

- No Priority 1 issues older than x days
- No Priority 2 issues older than x days
- Average work item age decreasing over time

Measurement: Current date minus issue creation date for all open issues.

6. Flow Efficiency

Definition: The percentage of time that work items are actively being worked on versus waiting.

Target: \> x% flow efficiency (industry standard is often 15-20%)

Measurement: Active work time (time spent in (~"workflow::refinement" + ~"workflow::in dev" + ~"workflow::in review")) divided by total lead time.

Required labels

Besides the GitLab product development flow labels, there are a number of additional required labels applied at any time to Epics, issues and merge requests (items):

- group::Self Managed - Items specific to, or authored by us.

There are also a number of additional required labels under certain scenarios:

- spike - Issues which primarily involve research to understand options and the breakdown of future deliverables. Spikes are often the first issue in a new Epic where the output defines additional issues and order of serial/parallel work.

In addition to the labels outlined above, see also workflow labels used during merge request review and labels used during issue triage.

Milestone Integration

Maintain alignment with GitLab milestone practices through:

- Tag any strict timeline issue with appropriate milestone during creation and keep it till completion, e.g. breaking changes
- Keep the milestone as Next 1-3 releases for all other issues when actively working on them
- Keep all other issues with milestone Backlog
- Tag all completed work with the milestone for recording
- Maintain all standard required labels

Blocked Issues Management Process

Blocked Issue Tracking

- ~"Workflow::blocked" Lane - Issues that cannot progress due to dependencies, waiting for input, or other impediments are moved to a dedicated "workflow::blocked" lane
- Blocker Documentation - Each blocked issue must include a comment describing:
 - What is blocking the issue
 - Who/what team is responsible for unblocking
 - Any actions taken to resolve the blockage
 - Expected timeframe for resolution (if known)

Blocked Issue Review (Daily Priority)

1. Daily Blocked Issues Triage
 - At the start of each day, the team reviews all blocked issues
 - Blocked issues are sorted by priority and duration of blockage
2. Escalation Path
 - Issues blocked for less than 2 days: Issue assignee actively follows up
 - Issues blocked for 2-5 days: Team sync/async discussion and EM involvement
 - Issues blocked for 5+ days: Escalation to appropriate managers/departments
3. Resolution Actions
 - Issue assignee is responsible for following up on each blocked issue
 - Document all actions taken to resolve the blockage
 - Consider workarounds or scope adjustments if blockage cannot be resolved quickly
4. Metrics and Reporting
 - Track percentage of issues blocked vs. flowing
 - Report on average time to resolve blockages
 - Identify patterns in blockers to address systemic issues
5. WIP Management for Blocked Issues
 - Blocked issues still count against workflow::in dev WIP limits
 - Team may temporarily adjust capacity allocation when multiple issues are blocked
 - Consider swarming on blocked issues to remove impediments

Unblocking Workflow

mermaid

flowchart TB

```
classDef normal fill:d4e8f6,stroke:3498db
classDef action fill:d4f6c8,stroke:2ecc71
classDef blocked fill:ffd4d4,stroke:e74c3c
```

```
IB[Issue Becomes Blocked]:::blocked --> |Move to| BL[workflow::blocked Lane]:::blocked
BL --> |Document| BC[Blocker Cause]:::action
BL --> |Set| ET[Expected Timeframe]:::action
```

```
BL --> |Daily Review| DR[Daily Blocked Issue Review]:::normal
DR --> |Can Proceed?| D{Decision}
```

```
D -->|Yes| UI[Unblock Issue]:::action
D -->|No| AE{Age Evaluation}
```

```
AE -->|< 2 days| FU[Team Member Follows Up]:::action
AE -->|2-5 days| TE[Team Discussion & EM Involvement]:::action
AE -->|> 5 days| ES[Escalate to Senior Management]:::action
```

```
FU --> DR
TE --> DR
ES --> DR
```

```
UI --> |Move back to original column| WF[Workflow Continues]:::normal
UI --> |Document resolution| DR[Document Resolution]:::action
```

SECTION: engineering/infrastructure-platforms/production-engineering/_index.md

Teams

Production Engineering consists of:

- 1. Foundations
- 1. Observability
- 1. Ops
- 1. Runners Platform

SECTION: engineering/infrastructure-platforms/production-engineering/foundations/_index.md

Mission

The mission of the Production Engineering Foundations team at GitLab is to build and evolve the networking infrastructure that powers GitLab SaaS while maintaining the stability of select core platform services. We focus on developing innovative networking solutions that scale with GitLab's growth, while ensuring our maintained services remain reliable and efficient.

Vision

The Foundations team's North Star consists of two pieces:

- 1. Excellence in networking infrastructure. We will drive GitLab's networking capabilities for GitLab forward by building scalable, secure, and efficient solutions. This includes evolving our edge services, load balancing, rate limiting, and network security to meet the growing demands of all GitLab platforms. Through centralized networking tooling and infrastructure, we create a foundation that supports GitLab's continued growth and innovation.
- 1. Sustainable toil and service maintenance. While toil is inherent in SRE work, we will adopt processes and policies that create an effective balance between automation and manual work. This approach ensures we can maintain our core infrastructure services reliably while keeping operational overhead minimal as GitLab grows. We strive for efficiency in both our day-to-day operations and our maintenance of essential platform services.

See our team impact overview for an overview of all we accomplished in 2024.

Responsibilities

Primary Areas of Ownership

The Foundations team's flagship focus is our networking infrastructure, which we actively work to improve and expand:

Networking Infrastructure

- Edge and ingress services
- Load balancing (HAProxy, Ingress)
- Web Application Firewall (WAF)
- DNS (AWS Route 53, Cloudflare)
- VPC management
- CDN (Cloudflare)
- Network security and access controls
- Service discovery (Consul)
- Rate limiting
 - Design and implementation of rate limiting services

Maintained Services

While these services are essential to GitLab's infrastructure, we focus on maintaining their stability and reliability rather than actively expanding their capabilities. Features and improvements will not be prioritized:

- SaaS K8s workloads. GitLab-com is owned by Delivery, Tanka and Helmfiles are often shared by other teams.
- Secrets Management (Vault)
- Ops.gitlab.net and Ops Runners.
- Teleport - Foundations manages KTLO work such as upgrades, but all other ownership is managed by CorpSec.

Getting Assistance

Should you require assistance from the Foundations team, please open an issue in the Production engineering tracker

Apply one of the following templates:

- For issues pertaining to Teleport connections, use the teleport-troubleshooting template
- For issues pertaining to Cloudflare, use the Cloudflare Troubleshooting template
- For anything else, use the request-foundations template
- We also have team handles that ping the full team
 - Slack: @infra-foundations
 - GitLab: @gitlab-org/production-engineering/foundations

We rotate triage duties weekly between team members as part of our interrupt rotation.

Priority Levels for Requests

Priority	Description	Engagement SLA
Production Engineering::1	Very urgent, blocking significant other work	Engage within 24 hours
Production Engineering::2	A blocker, but we have workarounds	Engage within 3 business days
Production Engineering::3	Not currently a blocker but will be soon	Engage within a week

| Production Engineering::4 | Not likely to be a blocker, this is a nice-to-have improvement or suggestion | Engage within a month |

> [!note]

> Due to the complexity of our systems, we cannot commit to a resolution-based SLA when you open an issue with a given priority level, because we don't know how long something will take to resolve, and there is often more information we need from the stakeholder or external customer, which takes time to obtain. Therefore, we can only commit to an SLA for how quickly we will engage with an issue and prioritize the work.

We endeavor to triage incoming requests twice per week.

> [!important]

> If you have an issue that is very urgent and requires immediate attention (i.e. Priority 1), please also send us a link to the request issue in our Slack channel: gfoundations.

Considerations for tooling and maintaining our services

- We support dogfooding when it makes sense to do so.
- We consider the scale at which we run gitlab.com and the additional engineering effort required to dogfood, weighed against the capacity and other priorities of the team.
- When considering external tooling, we look first to other open source projects that are part of CNCF
- Further evaluation criteria include:
 - Technical Fit - How well does it solve the problem we need it to solve?
 - Service Maturity - How well developed is the tool, does it have a proven track record of being used at scale?
 - Maintainability - Does it have strong community engagement, which will ensure it is well maintained for the foreseeable future? Complexity of keeping it up to date?
 - Cost - Licensing costs when applicable, weighed against the cost of building and maintaining the tooling ourselves. Engineering time required to implement new tooling and maintain it, weighed against the benefit it brings.
 - License - Identify the specific license under which the open-source project is distributed (GNU, MIT, ISC, etc.). This allows us to ensure compatibility with other tools' licensing, contribute to the project if needed in the future, and understand the restrictions and risks associated with a specific license.

Key Performance Indicators

KPIs for the team are under development in <https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues/24928>.

Team Members

{{< team-by-manager-slug "steve-abrams" >}}

Key Technical Skills

The Foundations Team must maintain a broad and diverse set of technical skills while also maintaining the ability to switch contexts frequently. Some of these technical skills include:

- Cloudnative Engineering - Proficiency in Kubernetes and the associated ecosystem of running cloudnative services.
- Infrastructure as Code - Proficiency in Chef and Terraform
- Network Systems - Understanding of network concepts and experience with our Edge stack (see Edge services above)

Common Links

- Foundations Top Level Epic
- Foundations team meeting agenda
- Foundations Roadmap
- gfoundations - work related discussions, external requests, etc
- gfoundationssocial - socializing and standups
- gfoundationsnotifications - automated MR notifications
- gfoundationsalerts - pipeline failures and service alerts

How We Work

Team Sync meetings

- We have weekly synchronous meetings in two different time zones to encourage discussion, give status updates, triage incoming requests, and connect as a team.
 - Agenda
 - Youtube playlist.

Calendar

- We have a internal team calendar for regular team meetings.
 - To subscribe to the calendar you can use Other Calendars > Subscribe to calendar with cfbda056c7fecb09a0d0d0801527838384fe862b80f2e36cdc24bca7b0d9d274f@group.calendar.google.com the ID.

Standup

We have Geekbot automated checkins on Mondays and Fridays in the gfoundations channel. Any question can be skipped by replying "-".

- Monday questions include:
 - What are you working on this week?
 - Do you have any blockers?
- Friday questions include:
 - What did you do this week?
 - Do you have any shout outs?
 - Anything else you'd like to share?

Retros

We have a quarterly async retro that aligns with the company fiscal quarters and OKRs. In addition to reflecting on our quarter in general, we take a set of actions to try the next quarter and revisit how successful they were in the next retro.

FY26Q1 Actions

- Define a KTLO/Backlog grooming process and try it out (see <https://gitlab.com/gitlab-com/gl-infra/production-engineering/-/issues/26338> for more details).
- Documenting things is part of our definition of done (issues are not closed if there are relevant docs/runbooks/etc to be updated).

Picking up work

We have three buckets of work:

1. Project work
1. External requests
1. General operations, also known as Keep the lights on (KTLO)

While we rely on the workflow-infra:: labels to denote the current state of any given issue, we rely on the Build Board to understand what specific issues are ready to be picked up.

In short, anyone on the team can find issues to work on in the Build Board.

When something is prioritized to be worked on, the Foundations Build label is applied so it is included on the board. With the exception of some external requests, issues being added should be in workflow-infra::Ready (clearly defined and ready for any team member to pick up).

We want the build board to be relatively small in size so it is easy to traverse and there is no confusion around what needs to be worked on. It can provide some options for people to find work, but also prevent key work from being missed.

Different people are responsible for adding the Foundations Build label to issues for each bucket of work:

- Project Work - DRIs of each project (epic) uses the Foundations Build label to communicate what the next issues needed to be picked up are.
- External requests - the Engineering manager or person on the interrupt rotation will add Foundations Build to issues that need attention. These will also include the unblocks others label to make it clear to people on the interrupt rotation which issues are external requests. The person on interrupt rotation can also actively triage incoming issues, adding the Foundations Build label when new issues are opened.
- KTLO - These issues will generally be added according to a general combination of due date/priority/severity/weight. Any team member should feel empowered to add KTLO issues to the board when they fit the criteria (prioritization makes sense based on the due date/priority/severity/weight).

Each team member is expected to have one KTLO issue ongoing at a time, treated as the next priority behind project work. Monthly we review which issues are a priority, based on what we need to do and what we want to improve. These issues will have the Foundations Next label to make them easily identifiable.

We trust everyone to be a Manager of One - owning not just their own work, but the team's work

- taking responsibility to define their own workflow that provides the right balance of work

for them to be most impactful to the team.

Prioritization of work

We use priority labels to prioritize our work. project work is automatically set considered Production Engineering::2 when it is ready to be worked on. As such it is given higher priority than most other work. This means that external requests will be worked in based on their own priority and impact, where only P1 and P2 issues will regularly interrupt project work.

Interrupt Rotation

The goal of the team member on interrupt rotation is to purposely plan to have interrupt work for a week so that the rest of the team can have less interrupt work. The schedule is available in pager duty.

If a team member is unavailable for more than 2 days of the week they are on triage, they should seek to trade weeks or find coverage for the days they are out.

The team member on rotation is responsible for:

- Checking the Foundations Build Board throughout the week and ensuring new issues related to Foundations:
 - Have the correct labels (Foundations::, Production Engineering::P, workflow-infra::, any other applicable category labels)
 - Have a weight.
 - Are assigned to an appropriate epic if applicable.
- Asking issue authors for more information when needed to understand the scope of work.
- Raising any Production Engineering::1 or Production Engineering::2 issues and ensuring they have a DRI and are being actively worked on.
- Completing any issues that come up that can be completed during the week. Weight 5 issues can be considered depending on the priority.
 - Anything weight 8 or larger (more than 1 week of work) should be made clear that it will need to be scheduled around other work.
- Responding to alerts in gfoundationsalerts.
- Working on Renovate MRs posted in gfoundationsnotifications.
 - Patch updates with passing pipelines should be merged.
 - Minor updates should have their changelogs be reviewed before merging.
 - Major updates should be looked at closely to understand breaking changes before considering merging.
- Responding to Terraform drifts related to Foundations services posted in infra-terraform-alerts.
- Responding to any Access Requests that need Foundations provisioning.

Project Management

The Foundations team top level Epic can be found here
We follow Platforms Project Management practices as outlined in the Handbook.
Below builds on top of those guidelines.

OKRs

- For Objectives and Key Results, we align with Platforms guidance for creation and structure.

Epics / Projects

In addition to the format described in the platforms project management page, these optional sections may be helpful.

markdown

Administrative

<!-- A copy paste section for creating child epics/issues, ensuring that they relate to the current epic and have the correct labels -->

\\\`

/epic [current epic]

/labels ~"group::Production Engineering" ~"team::Foundations" ~"workflow-infra::Triage"

~"Production Engineering::P2" ~"Foundations::Project work"

\\\`

References

<!-- Links to related OKRs, Epics or issues, external resources etc -->

Demos

| Demo Date | Demo Link | Highlights |

|-----|-----|-----|

Decision log

<!-- A collapsible section to aggregate any decisions made along the way. Be sure to include "why" in addition to "what". -->

{{% details summary="Log" %}}

{{% details summary="date" %}}

[decision taken and why]

{{% /details %}}

{{% /details %}}

- Apply any applicable service labels.

- When changing the health status to anything other than "On track", give context for why in the weekly status update, including the plan to get back on track when possible.

Issues

Most issues for our team are tracked primarily in the Production Engineering project. Issues should be updated whenever significant work occurs. New issues are expected to:

- Link to a related Epic.
- Include the following Labels (update the priority as needed):

```
text
/labels ~"group::Production Engineering" ~"team::Foundations" ~"workflow-infra::Triage"
~"Production Engineering::P4"
```

- A Foundations label that aligns with the bucket of work it fits best in: Foundations::KTLO, Foundations::Requests, Foundations::Project Work, ...

```
> [!important]
> Access Requests issues use the Foundations::Todo, Foundations::Doing and Foundations::Done
labels.
```

- If there is a service label that is applicable, also apply that.

Issue weighting

We dogfood the weight feature in issues. We use a modified Fibonacci sequence (the numbers match roughly with the number of days) and align our weights to t-shirt sizes to make them easy to talk about:

- Fib 1 (XS) - A few hours of work. Consider if the work should just be completed now depending on competing priorities.
- Fib 2 (S) - A day or two of work.
- Fib 5 (M) - Up to a week of work.
- Fib 8 (L) - Over a week of work. This issue should likely be broken down into smaller pieces.
- Fib 13 (XL) - Several weeks or more of work. This issue will need to be broken down into smaller pieces or promoted to an epic.

SECTION: engineering/infrastructure-platforms/production-engineering/observability/_index.md

Observability encompasses the technical elements responsible for metrics, logging, and tracing, along with the tools and processes that leverage these components.

Mission

Our mission is to deliver and maintain a world-class observability offering and frictionless operational experience for team members at GitLab.

Common Links

```
|
|
|-----|-----
|-----|-----
|-----|-----
```

Workflow	Team workflow
GitLab.com	@gitlab-org/production-engineering/observability
Issue Trackers	Observability Tracker Tamland
Team Slack Channels	gobservability - Team channel infrastructureplatformsocial - Social channel
Information Slack Channels	infrastructure-lounge (Infrastructure Group Channel), incidents (Incident Management), ginfraobservabilityalerts (Observability Slack Alerts), alerts-general (SLO alerting), mechsympalerts (Mechanical Sympathy Alerts)
Documentation	Documentation Hub

Team Members

The following people are members of the Observability team:

```
{{< team-by-manager-slug "liam-m" >}}
```

The team is located all over the world in different timezones.

Technical principles, goals and responsibilities

Please see the Technical Blueprint for details on our principles and goals.

The following gives an overview of our scope and ownership.

1. Monitoring fundamentals
 1. Metrics stack
 1. Logging stack
1. Error budgets
 1. Ownership of concept and implementation
 1. Delivery of monthly error budget report
1. Capacity planning
 1. Triage rotation for .com
 1. Operational aspects for GitLab Dedicated capacity planning
 1. Developing Tamland, the forecasting tool
 1. Capacity reporting for GitLab Dedicated
1. Service Maturity model which covers GitLab.com's production services.
1. GitLab.com availability reporting: Provide underlying data and aggregate numbers

Documentation

We recognize the need to provide technical documentation for teams using our observability services and platforms, as well as for our team's internal use.

Historically, we've provided reference documentation within the projects we own or contribute to, or in the runbooks project.

As these projects are scattered around, it is rather difficult to discover the various pieces of relevant documentation for our users.

As we reshape our documentation, we follow along with the following idea and principles:

1. The Infrastructure Observability Documentation Hub is the entrypoint for any observability related documentation we provide.

1. Carefully crafted documentation is a core product for the observability platform, not an afterthought.

1. We think of the documentation hub as a way to communicate about observability interfaces we offer as a platform with everyone in Engineering.

1. We strive to provide documentation like guides and tutorials targeted for specific audiences - in addition to reference documentation.

1. We use the documentation hub also to explain concepts and architecture for our team's internal use to create a common understanding and empower everyone to contribute in all areas.

Where do we keep different types of documentation?

There are different types of documentation, which belong in different places.

What?	Where?	
-----	-----	
Team organisation, processes and other team level agreements	GitLab Handbook (this page)	
Technical reference documentation for standalone projects	On the project itself, and linked to the Documentation Hub	
How do we at GitLab make use of the projects we maintain?	Documentation Hub	
How does our GitLab specific architecture look like?	Documentation Hub	
Tutorials, guides, FAQs and conceptual explanations	Documentation Hub	

Documentation outside the Documentation Hub should be linked from it (that's why we call it a hub) and vice versa, to help increase discoverability.

This recognizes the need to ship reference documentation with the respective project, as we would expect to see for any project (whether open source or not).

The benefit here is that a change in functionality can also update reference documentation in the same merge request.

On the other hand, how we make particular use of these projects in our stack is too specific to ship with the project itself.

Often, we want to understand the bigger picture and how projects play together.

This is out of scope for technical documentation that ships with a certain project itself and hence we put this information on the documentation hub instead.

For our internal use, we use the documentation hub to help us reason about the services we own and how we operate them.

We expect this helps everyone on the team and helps us gather a common understanding as we have different roles and perspectives on the team.

A recommended read on different types of documentation and how to organize it is the Divio Documentation System.

How do we create documentation?

As we reshape and build documentation, the documentation hub benefits from each and all contributions:

1. Explain existing concepts
1. Link together existing documentation
1. Consolidate existing documentation and move in the right places
1. Writing and graphics on system architecture and operational principles

We aspire to create and maintain documentation as a primary citizen and similar to the Handbook First mindset.

For example, instead of answering specific questions from team members individually (for example on Slack), we can take this as an opportunity to write a piece of documentation and ask them to review and work with that.

Indicators

The group is an owner of several performance indicators that roll up to the Infrastructure department indicators:

1. Service Maturity model which covers GitLab.com's production services.
1. Capacity Planning uses capacity warnings to prevent incidents.

These are combined to enable us to better prioritize team projects.

An overly simplified example of how these indicators might be used, in no particular order:

Service Maturity - provides detail on how trustworthy the data we received from observability stack in relation to the service; the lower the level the more focus we need to improve the service observability

Capacity Planning - Provides a forecast for a specific service

Between these different signals, we have a relatively (im)precise view into the past, present and future to help us prioritise scaling needs for GitLab.com.

Provisioned Services

The team are responsible for provisioning access to the services listed below, as per the techstack.yml file.

1. Kibana is accessed through Okta. Team members need to be in either of the following Okta groups: gl-engineering (entire Engineering department); okta-kibana-users. The latter group is used to manage access for team members outside of Engineering on an ad-hoc basis (context). Team members should be (de)provisioned through an Access Request (example). If the access request is approved, the provisioner should add the user to this group, which will then automatically sync to its namesake group in Okta.

1. Elastic Cloud is for administrative access to our Elastic stack. The login screen is available [here](#) and access is through Google SSO. Team members should be (de)provisioned through an Access Request ([example](#)). If approved, the provisioner can add/remove members on the membership page with appropriate permissions to the instances they require access to.

1. Grafana is accessed through Okta. The login screen is available [here](#). Any GitLab team member can access Grafana. Provisioning and deprovisioning is handled through Okta.

How we work

We default to working inline with the GitLab values and by following the processes of the wider SaaS Platforms section and Scalability group.

In addition to this, listed below are some processes that are specific, or particularly important, to how we work in Observability.

Roadmap

We transparently prioritize our Roadmap in this [issue](#).

Project Sizing

Projects will be staffed by at least three engineers from the team, and preferably from both SRE and BE roles. This will allow us to share knowledge and keep projects moving forward when people are unavailable.

Issue management

While we mainly operate from the observability issue tracker, there are other projects under the gl-infra group team members are working on.

Hence we strive to use group-level labels and boards to get the entire picture.

Labels

All issues pertaining to our team have the ~"team::Observability" label.

All issues that are within scope of current work have a ~board::build or ~board::planning label.

This is a measure to cut through noise on the tracker and allows us to get a view on what's currently important to us.

See Boards below on how this is being used.

All issues require either a Service label or the team-tasks, discussion, or capacity planning labels.

Assignees

We use issue assignments to signal who is the DRI for the issue.

We expect issue assignees to regularly update their issues with the status, and to be as explicit as possible at what has been done and what still needs to be done.

We expect the assignee of an issue to drive the issue to completion.

The assignee status typically expresses, that the assigned team member is currently actively working on this or planning to come back to it relatively soon.

We unassign ourselves from issues we are not actively working on or planning to revisit in a few days.

Boards

The Observability team's issue boards track the progress of ongoing work.

We use issue boards to track the progress of planned and ongoing work.

Planning	Building
Planning Board	Build Board
Issues where we are investigating the work to be done.	Issues that will be built next, or are actively in development.

Retrospectives

A team-level retrospective issue is created every 6 weeks, allowing the team to regularly reflect and to encourage a culture of continuous improvement. The creation of the retrospective issue is the responsibility of the Engineering Manager. You can find retrospectives [here](#).

Updates in Slack

We are using GeekBot for weekly updates, which go to the [gobservability](#) channel.

When posting updates, consider providing enough context (e.g. through links) so that interested team members are able to dive in on their own (low context).

Cost Management

For details on the daily operational costs of our observability services refer to the [Cost of Observability Stack](#) documentation. This resource includes access instructions and cost breakdowns.

History and Accomplishments

This section contains notes on team history and our accomplishments, but is not meant to be exhaustive.

2024-02, Capacity planning: Proactive investigation of postgres CPU spike seen in saturation forecast uncovered a database design issue

2024-03, Capacity planning: Tamland predicted redis CPU saturation which led to Practices proactively scaling Redis (slides)

2024-05, Metrics: The migration from Thanos to Mimir was completed which brought significant improvements in metrics accuracy and dashboard performance.

Year-in-Review Issues

2024

SECTION: engineering/infrastructure-platforms/production-engineering/observability/capacity_planning.md

We maintain and improve the Capacity Planning process that is described in the Infrastructure Handbook. This is a controlled activity covered by SOC 2. Please see this issue for further details

The goal of this process is to predict and prevent saturation incidents on GitLab.com.

Issues are kept in the capacity planning issue tracker. Where an issue is needed to improve metrics to support this process, we raise an issue in the